

2.2. Operadores y estructuras de control en Dart



DESARROLLO DE APLICACIONES WEB MULTIPLATAFORMA NIVEL 3.

UNIDAD 2:

FUNDAMENTOS DE DART

Contenido

Introducción.....	4
Tipos de operadores en Dart	4
Operadores aritméticos:	4
Operadores relacionales:.....	5
Operadores lógicos:.....	5
Operadores de asignación:.....	5
Operadores de incremento y decremento:.....	6
Operadores de igualdad	6
Operadores de tipo:	7
Ejemplos prácticos con operadores en Dart.....	8
Ejemplo 1: Calcular el área de un triángulo	8
Ejemplo 2: Determinar si un año es bisiesto.....	8
Ejemplo 3: Buscar el mayor de tres números	9
Estructuras de control en Dart.....	11
Instrucciones if y else	11
Instrucción switch	12
Bucles for.....	14
Bucles while	14
Ejemplos prácticos con estructuras de control en Dart	15
Ejemplo 1: Calcular la suma de los números pares hasta 100	15
Ejemplo 2: Mostrar un menú de opciones y ejecutar la acción seleccionada	15
Consideraciones adicionales.....	17
Precedencia de operadores.....	17
Expresiones condicionales anidadas.....	17
Bucles anidados.....	18

Errores comunes en el uso de operadores y estructuras de control.....	19
Recursos adicionales.....	20

Introducción

Los operadores son símbolos que se utilizan para realizar operaciones con datos en un programa Dart.

Los operadores juegan un papel fundamental en la programación Dart, ya que permiten:

- Realizar cálculos aritméticos: sumar, restar, multiplicar, dividir, etc.
- Comparar datos: verificar si un dato es mayor, menor o igual que otro.
- Combinar expresiones lógicas: crear condiciones más complejas para controlar el flujo del programa.
- Asignar valores a variables: almacenar datos en variables para su uso posterior.
- Manipular datos: incrementar o decrementar valores, comprobar tipos de datos, etc.

En resumen, los operadores son herramientas esenciales para construir la lógica y funcionalidad de cualquier programa Dart.

Tipos de operadores en Dart

Dart ofrece una amplia variedad de operadores para realizar diferentes tareas. A continuación, se detallan algunos de los tipos de operadores más comunes:

Operadores aritméticos:

Son los siguientes:

- +: Suma
- -: Resta
- *: Multiplicación
- /: División
- %: Módulo (residuo de la división)
- ++: Incremento
- --: Decremento

Ejemplo:

- `int resultado = 10 + 5; // resultado = 15`
- `int diferencia = 20 - 3; // diferencia = 17`

- `int producto = 4 * 6; // producto = 24`
- `int division = 12 / 3; // division = 4`
- `int residuo = 18 % 5; // residuo = 3`

Operadores relacionales:

Son los siguientes:

- `==`: Igualdad
- `!=`: Desigualdad
- `<`: Menor que
- `<=`: Menor o igual que
- `>`: Mayor que
- `>=`: Mayor o igual que

Ejemplo:

- `bool esIgual = 10 == 10; // esIgual = true`
- `bool esDistinto = 7 != 12; // esDistinto = true`
- `bool esMenor = 5 < 8; // esMenor = true`
- `bool esMenorIgual = 15 <= 15; // esMenorIgual = true`
- `bool esMayor = 22 > 18; // esMayor = true`
- `bool esMayorIgual = 30 >= 30; // esMayorIgual = true`

Operadores lógicos:

Son los siguientes:

- `&&`: Y (AND)
- `||`: O (OR)
- `!`: No (NOT)

Ejemplo:

- `bool esAprobado = nota >= 60 && asistencia >= 80; // esAprobado = true (si nota y asistencia son >= 60 y 80, respectivamente)`
- `bool tieneDescuento = edad < 18 || esJubilado; // tieneDescuento = true (si edad es < 18 o esJubilado es true)`
- `bool esVerdadero = !falso; // esVerdadero = true (NOT de falso es true)`

Operadores de asignación:

Son los siguientes:

- `=`: Asignación simple

- +=: Asignación con suma
- -=: Asignación con resta
- *=: Asignación con multiplicación
- /=: Asignación con división
- %=: Asignación con residuo

Ejemplo:

- `int numero = 5; // Asignación simple`
- `numero += 3; // Asignación con suma (numero = 8)`
- `numero -= 2; // Asignación con resta (numero = 6)`
- `numero *= 4; // Asignación con multiplicación (numero = 24)`
- `numero /= 3; // Asignación con división (numero = 8)`
- `numero %= 2; // Asignación con residuo (numero = 0)`

Operadores de incremento y decremento:

Son los siguientes:

- ++: Incremento
- --: Decremento

Ejemplo:

```
int contador = 0;  
  
contador++; // contador = 1  
  
contador--; // contador = 0
```

Operadores de igualdad

- ==: Igualdad estricta (comprueba valor y tipo)
- !=: Desigualdad estricta (comprueba valor y tipo)

Ejemplo:

```
String texto1 = "Hola";  
  
String texto2 = "Hola";  
  
  
  
bool sonIguales = texto1 == texto2; // sonIguales = true (compara solo  
el valor)
```

```
bool sonIgualmenteEstrictos = texto1 === texto2; //  
sonIgualmenteEstrictos = true (compara valor y tipo)
```

```
int numero1 = 10;  
int numero2 = 10.0;
```

```
bool sonIgualesNumeros = numero1 == numero2; //  
sonIgualesNumeros = true (compara solo el valor)
```

```
bool sonIgualmenteEstrictosNumeros = numero1 === numero2; //  
sonIgualmenteEstrictosNumeros = false (compara valor y tipo, son de  
diferente tipo: int)
```

Operadores de tipo:

- is: Verificar si un objeto es de un tipo específico
- as: Convertir un objeto a un tipo específico

Ejemplo:

```
class Persona {  
  
    String nombre;  
  
    int edad;  
  
    Persona(this.nombre, this.edad);  
  
}
```

```
Persona persona1 = Persona("Juan", 30);
```

```
bool esPersona = persona1 is Persona; // esPersona = true
```

```
Persona? persona2 = persona1 as Persona?; // persona2 = persona1  
(conversión segura)
```

```
String nombrePersona = persona1.nombre as String; //  
nombrePersona = "Juan" (conversión segura)
```

```
dynamic dato = "Hola";
```

```
String? texto = dato as String?; // texto = "Hola" (conversión segura)
```

```
if (dato is String) {  
    print("El dato es una cadena de texto");  
} else {  
    print("El dato no es una cadena de texto");  
}
```

Ejemplos prácticos con operadores en Dart

A continuación, se presentan algunos ejemplos prácticos adicionales que muestran el uso de operadores en Dart:

Ejemplo 1: Calcular el área de un triángulo

```
double base = 10.0;  
double altura = 5.0;  
double area = (base * altura) / 2;  
  
print("El área del triángulo es: $area");
```

Ejemplo 2: Determinar si un año es bisiesto

```
int año = 2024;
```

```
bool esBisiesto = (año % 4 == 0) && (año % 100 != 0) || (año % 400 == 0);
```

```
if (esBisiesto) {  
    print("$año es un año bisiesto");  
} else {  
    print("$año no es un año bisiesto");  
}
```

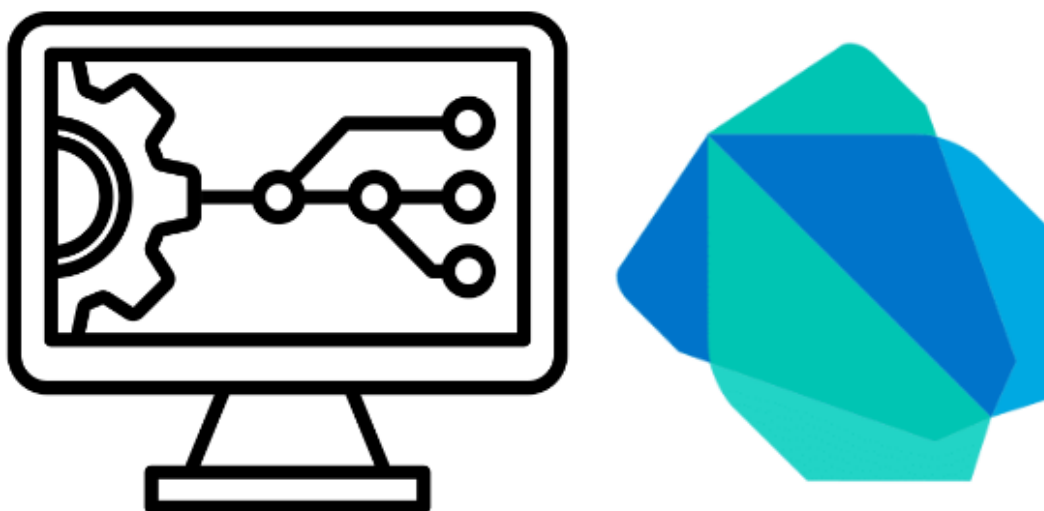
Ejemplo 3: Buscar el mayor de tres números

```
int numero1 = 12;  
int numero2 = 18;  
int numero3 = 15;  
  
int mayor = numero1;  
  
if (numero2 > mayor) {  
    mayor = numero2;  
}  
  
if (numero3 > mayor) {  
    mayor = numero3;  
}  
  
print("El número mayor es: $mayor");
```

Usa el código con precaución.

Estos son solo algunos ejemplos básicos. A medida que se avanza en el aprendizaje de Dart, se encontrarán con casos de uso más complejos que requieren el uso combinado de diferentes operadores y estructuras de control que veremos en el siguiente epígrafe.

Estructuras de control en Dart



Las estructuras de control son bloques de código que permiten controlar el flujo de ejecución de un programa Dart.

Las estructuras de control juegan un papel fundamental en la programación Dart, ya que permiten:

- Ejecutar código condicionalmente: decidir qué código se ejecuta en función de una condición.
- Repetir código un número determinado de veces: realizar tareas repetitivas de manera eficiente.
- Ejecutar código de forma iterativa: recorrer conjuntos de datos y procesar cada elemento.
- Estructurar el código de manera lógica: organizar el código de forma clara y comprensible.

Instrucciones if y else

La instrucción if permite ejecutar un bloque de código si se cumple una condición específica. La instrucción else se utiliza para ejecutar un bloque de código alternativo si la condición del if no se cumple.

Sintaxis:

```
if (condicion) {  
    // Bloque de código a ejecutar si la condición es verdadera
```

```
} else {  
    // Bloque de código a ejecutar si la condición es falsa  
}  
  
Usa el código con precaución.
```

Ejemplo:

```
int edad = 18;  
  
if (edad >= 18) {  
    print("Eres mayor de edad");  
} else {  
    print("Eres menor de edad");  
}
```

En resumen, este código declara una variable edad y le asigna el valor 18. Luego, utiliza una estructura condicional if-else para determinar si la persona representada por la variable edad es mayor o menor de edad, e imprime un mensaje correspondiente en la consola.

Instrucción switch

La instrucción switch permite ejecutar diferentes bloques de código en función de una variable que toma distintos valores. Es similar a una serie de instrucciones if anidadas.

Sintaxis:

```
switch (variable) {  
    case valor1:  
        // Bloque de código a ejecutar si variable es igual a valor1  
        break;  
    case valor2:
```

```
// Bloque de código a ejecutar si variable es igual a valor2  
break;  
  
...  
  
default:  
  
    // Bloque de código a ejecutar si variable no es igual a ninguno de  
    los valores anteriores  
  
}
```

Ejemplo:

```
String dia = "Lunes";  
  
switch (dia) {  
    case "Lunes":  
        print("Hoy es lunes");  
        break;  
    case "Martes":  
        print("Hoy es martes");  
        break;  
    case "Miércoles":  
        print("Hoy es miércoles");  
        break;  
    ...  
    default:  
        print("No conozco ese día");  
}
```

En resumen, este código declara una variable `dia` con el valor "Lunes", utiliza una instrucción `switch` para determinar el día de la semana correspondiente y muestra un mensaje en la consola según el día.

Bucles for

Los bucles `for` permiten ejecutar un bloque de código un número determinado de veces.

Sintaxis:

```
for (inicializacion; condicion; actualizacion) {  
    // Bloque de código a ejecutar  
}
```

Ejemplo:

```
for (int i = 0; i < 5; i++) {  
    print("Número: $i");  
}
```

En resumen, este código utiliza un bucle `for` para iterar desde el 0 al 4 (inclusive), imprimiendo en la consola el mensaje "Número: " seguido del valor actual de la variable `i` en cada iteración.

Bucles while

Los bucles `while` permiten ejecutar un bloque de código mientras se cumpla una condición.

Sintaxis:

```
while (condicion) {  
    // Bloque de código a ejecutar  
}
```

Ejemplo:

```
int numero = 1;
```

```
while (numero <= 10) {  
    print("Número: $numero");  
    numero++;  
}
```

En resumen, este código declara una variable `numero` e inicializa su valor en 1. Luego, utiliza un bucle `while` que se ejecuta mientras `numero` sea menor o igual que 10. Dentro del bucle, se imprime el valor actual de `numero` en la consola y se incrementa su valor en 1. Esto hace que el bucle imprima los números del 1 al 10.

Ejemplos prácticos con estructuras de control en Dart

A continuación, se presentan algunos ejemplos prácticos adicionales que muestran el uso de estructuras de control en Dart:

Ejemplo 1: Calcular la suma de los números pares hasta 100

```
int sumaPares = 0;  
  
for (int i = 2; i <= 100; i += 2) {  
    sumaPares += i;  
}  
  
print("La suma de los números pares hasta 100 es: $sumaPares");
```

Ejemplo 2: Mostrar un menú de opciones y ejecutar la acción seleccionada

```
void main() {  
    print("Menú de opciones:");  
    print("1. Calcular el factorial de un número");  
    print("2. Verificar si un número es primo");  
    print("3. Salir");  
}
```

```
int opcion = int.parse(readLine()); // Leer la opción del usuario
```

```
switch (opcion) {
```

```
  case 1:
```

```
    calcularFactorial();
```

```
    break;
```

```
  case 2:
```

```
    verificar
```

Consideraciones adicionales

A continuación, te presentamos una serie de consideraciones adicionales asociadas al uso de operadores y estructuras de control:

Precedencia de operadores

La precedencia de operadores define el orden en que se evalúan las operaciones aritméticas, lógicas y relacionales en una expresión. Es importante comprender la precedencia de operadores para escribir código claro y evitar errores.

En Dart, la precedencia de operadores sigue la siguiente regla:

- Operadores de paréntesis: Los paréntesis tienen la mayor precedencia. Se utilizan para agrupar operaciones y forzar un orden de evaluación específico.
- Operadores unarios: Los operadores unarios, como -, !, ++ y --, se evalúan de derecha a izquierda.
- Operadores de multiplicación y división: Los operadores de multiplicación (*) y división (/) se evalúan de izquierda a derecha.
- Operadores de suma y resta: Los operadores de suma (+) y resta (-) se evalúan de izquierda a derecha.
- Operadores de comparación: Los operadores de comparación (==, !=, <, >, <=, >=) se evalúan de izquierda a derecha.
- Operadores lógicos: Los operadores lógicos (&&, ||, !) se evalúan de izquierda a derecha.

Ejemplo:

```
int resultado = 2 + 3 * 4; // Resultado: 14
```

```
// La multiplicación se evalúa antes que la suma (2 + (3 * 4))
```

Expresiones condicionales anidadas

Las expresiones condicionales anidadas permiten crear estructuras lógicas más complejas dentro de instrucciones if o switch. Esto permite evaluar múltiples condiciones y ejecutar diferentes bloques de código en función de los resultados de las evaluaciones.

Ejemplo:

```
int edad = 18;

String pais = "España";

if (edad >= 18) {
    if (pais == "España") {
        print("Eres mayor de edad y español");
    } else {
        print("Eres mayor de edad, pero no español");
    }
} else {
    print("Eres menor de edad");
}
```

En este ejemplo, se anidan dos instrucciones if:

- La primera instrucción if verifica si edad es mayor o igual que 18.
- Si la primera condición es verdadera, se ejecuta la segunda instrucción if, que verifica si pais es igual a "España".

Bucles anidados

Los bucles anidados permiten ejecutar un bucle dentro de otro bucle. Esto puede ser útil para recorrer estructuras de datos bidimensionales o realizar tareas repetitivas de forma más compleja.

Ejemplo:

```
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 5; j++) {
        print("Fila: $i, Columna: $j");
    }
}
```

En este ejemplo, se anidan dos bucles for:

- El bucle exterior recorre las filas (de 0 a 2).
- Para cada fila, el bucle interior recorre las columnas (de 0 a 4).

Errores comunes en el uso de operadores y estructuras de control

- No comprender la precedencia de operadores: Esto puede llevar a resultados inesperados en las expresiones.
- Olvidas usar paréntesis: Los paréntesis son esenciales para controlar el orden de evaluación de las operaciones.
- Condiciones incorrectas: Asegúrate de que las condiciones en las instrucciones if y switch sean correctas y reflejen la lógica deseada.
- Bucles infinitos: Los bucles sin una condición de salida adecuada pueden causar que el programa se ejecute sin fin.
- Errores de tipo de datos: Verifica que los operadores y expresiones se utilicen con tipos de datos compatibles.

Recursos adicionales

Documentación oficial:

- Documentación oficial de Dart: <https://dart.dev/guides>
- Documentación oficial de Dart sobre operadores: <https://dart.dev/language/operators>
- Documentación oficial de Dart sobre estructuras de control: <https://www.javatpoint.com/dart-control-flow-statement>

Tutoriales

- Dart: <https://dart.dev/tutorials>
- Operadores y estructuras de control en Dart:
 - <https://www.freecodecamp.org/news/tag/dart/>
 - <https://www.codecademy.com/resources/docs/dart>
 - <https://www.geeksforgeeks.org/introduction-to-dart-programming-language/>

Comunidad y foros:

- Foro de la comunidad de Dart: <https://github.com/dart-lang?language=dart>
- Stack Overflow para Dart: <https://stackoverflow.com/questions/tagged/dart>

¡Felicidades por completar la unidad didáctica sobre Operadores y estructuras de control en Dart!

En esta unidad, has aprendido los conceptos fundamentales para controlar el flujo de ejecución de tu programa en Dart. Has visto cómo utilizar operadores aritméticos, lógicos y relacionales para realizar cálculos y evaluar condiciones. También has aprendido a utilizar estructuras de control como if, switch, for y while para ejecutar bloques de código en función de diferentes escenarios.

Es importante recordar que la práctica es fundamental para dominar estos conceptos. Te animo a que continúes practicando lo que has aprendido escribiendo código Dart, experimentando con diferentes ejemplos y resolviendo problemas. También puedes buscar recursos adicionales como tutoriales, cursos y documentación para profundizar en tu conocimiento.

Estoy seguro de que con dedicación y esfuerzo, podrás convertirte en un programador Dart experto.

¡Te deseo mucho éxito en tu camino de aprendizaje!

¡Hasta la próxima!